

Replit AI Agent — FTA Tax Accounting Software Accreditation Implementation

CONTEXT

You are working on the InvoLinks.ae codebase (FastAPI + React + PostgreSQL/Neon). This is a live invoicing platform. Your job is to implement missing FTA (UAE Federal Tax Authority) Tax Accounting Software accreditation requirements.

CRITICAL RULES — READ BEFORE TOUCHING ANY CODE

1. **DO NOT break any existing functionality.** Every existing feature must continue to work exactly as before after your changes.
 2. **Before modifying any existing file,** read it fully first. Understand what it does, then make the minimum necessary change.
 3. **All new DB changes must be additive** — new tables, new columns with defaults. Never rename, drop, or alter the type of an existing column.
 4. **All new API endpoints must be additive** — do not modify existing endpoint signatures, response shapes, or status codes.
 5. **Test after every change.** After each implementation block, verify:
 - Existing invoice creation still works
 - Existing FAF export still downloads
 - Existing auth/login/MFA still works
 - Existing role permissions still enforced
 6. **Use database migrations** for every schema change — never alter tables directly.
 7. **Do not change existing UI pages** unless the task explicitly requires adding to them. Add new pages/components; do not restructure existing ones.
 8. **Preserve all existing API contracts** — no field renames, no removed fields in responses, no changed HTTP methods on existing routes.
-

IMPLEMENTATION TASKS

Implement the following in the exact order listed. Complete each task fully before starting the next. Each task includes what to build, which files are affected, and the acceptance criteria.

TASK 1 — Audit Trail Table (P1-4)

Why first: Every subsequent task writes financial data. The audit trail must exist before those writes happen so all changes are captured from day one.

What to build:

- New DB table: `audit_log`
 - `id` (UUID, PK)
 - `entity_type` (VARCHAR) — e.g. "invoice", "company", "user", "expense"
 - `entity_id` (UUID)
 - `action` (VARCHAR) — "CREATE", "UPDATE", "DELETE", "CANCEL", "STATUS_CHANGE"
 - `field_name` (VARCHAR, nullable) — the specific field that changed
 - `old_value` (TEXT, nullable)
 - `new_value` (TEXT, nullable)
 - `changed_by_user_id` (UUID, FK to users)
 - `changed_by_email` (VARCHAR) — denormalized for audit read performance
 - `ip_address` (VARCHAR, nullable)
 - `user_agent` (VARCHAR, nullable)
 - `timestamp` (TIMESTAMPTZ, default NOW())
 - `company_id` (UUID, FK to companies)
- New utility: `utils/audit_utils.py`
 - `log_audit(db, entity_type, entity_id, action, changed_by_user_id, changed_by_email, company_id, field_name=None, old_value=None, new_value=None, ip_address=None, user_agent=None)`

- `log_field_changes(db, entity_type, entity_id, old_obj_dict, new_obj_dict, changed_by_user_id, changed_by_email, company_id)` — diffs two dicts and logs one row per changed field
- Hook `log_audit()` into these existing operations (add calls without changing existing logic):
 - Invoice created → log CREATE
 - Invoice status changed → log STATUS_CHANGE (old_status → new_status)
 - Invoice cancelled → log CANCEL
 - Company settings updated → log UPDATE with field-level diff
 - User created/invited → log CREATE
 - User role changed → log UPDATE
 - Expense created/updated → log CREATE/UPDATE
- New API endpoint (read-only, admin only):
 - `GET /audit-log?entity_type=&entity_id=&from_date=&to_date=&page=&limit=`
 - Returns paginated audit log entries for the company

Acceptance criteria:

- Creating an invoice generates an audit_log row
- Cancelling an invoice generates an audit_log row with old/new status
- Existing invoice creation, cancellation, and listing endpoints return identical responses as before
- `GET /audit-log` returns results filtered by `company_id` of the requesting user

TASK 2 — Password Security Controls (P1-5)

What to build:

Schema changes (additive only):

- Add to `users` table:
 - `must_change_password` (BOOLEAN, default FALSE)

- `password_changed_at` (TIMESTAMPTZ, nullable)
- `failed_login_attempts` (INTEGER, default 0)
- `locked_until` (TIMESTAMPTZ, nullable)
- New table: `password_history`
 - `id` (UUID, PK)
 - `user_id` (UUID, FK to users)
 - `password_hash` (VARCHAR)
 - `created_at` (TIMESTAMPTZ, default NOW())

Backend changes:

- Password validation function `validate_password_strength(password: str) :`
 - Minimum 8 characters
 - At least 1 uppercase letter
 - At least 1 lowercase letter
 - At least 1 digit
 - At least 1 special character (!@#\$%^&*...)
 - Raise HTTP 400 with clear message if any rule fails
- Apply `validate_password_strength()` to: registration, password reset, and password change endpoints — do not change the endpoint signatures, just add the validation call inside
- Password history check: on password change, verify new password does not match last 5 hashes in `password_history`. Store new hash in `password_history` after successful change.
- `must_change_password = True` set server-side when:
 - Admin invites a user (temp password generated)
 - Admin resets another user's password
- On login: if `must_change_password = True`, return HTTP 200 with body `{"must_change_password": true, "temp_token": "..."} — existing login flow for normal users must be completely unchanged`

- Account lockout: after 5 consecutive failed login attempts, set `locked_until = NOW() + 15 minutes`. Return HTTP 423 with message. Reset `failed_login_attempts = 0` on successful login.

Frontend changes:

- Add password strength indicator component to registration and password change forms (shows which rules pass/fail in real time)
- If login response contains `must_change_password: true`, redirect to a `/change-password` page before allowing access to the app
- Do not change the layout or flow of the existing login page

Acceptance criteria:

- Existing users with valid passwords can still log in without disruption
- Weak password on registration returns a clear error
- Invited user is forced to change password before accessing the dashboard
- After 5 failed logins, account locks for 15 minutes

TASK 3 — Lock Posted Invoices (P2-1)

What to build:

- In the invoice UPDATE endpoint (existing), add a guard at the top:

```
if invoice.status in [InvoiceStatus.ISSUED, InvoiceStatus.PAID]:
    raise HTTPException(
        status_code=409,
        detail="Cannot edit an issued or paid invoice. Issue a credit note ins
    )
```

- In the invoice DELETE/CANCEL endpoint, if status is PAID:
 - Already blocked — verify this guard exists and keep it
 - If status is ISSUED: allow cancellation only if a corresponding credit note exists referencing this invoice. Otherwise return HTTP 409 with message "Issue a credit note before cancelling this invoice."
- Add frontend guard: if user tries to edit an ISSUED or PAID invoice, show a clear modal: "This invoice has been issued. To make corrections, please create a Credit Note." with a

button linking to the credit note creation flow.

- Do not change behaviour for DRAFT invoices — they must remain fully editable.

Acceptance criteria:

- DRAFT invoices: edit, save, delete all work as before
 - ISSUED invoice: edit attempt returns 409; frontend shows modal
 - PAID invoice: edit attempt returns 409
 - Existing invoice status transitions (DRAFT→ISSUED, ISSUED→PAID) unchanged
-

TASK 4 — Debit Note Support (P2-2)

What to build:

- Add `invoice_type = "383"` as a valid enum value (Debit Note) alongside existing "380" (Invoice) and "381" (Credit Note)
- DB: add `debit_note_reason` VARCHAR column to invoices table (nullable, with default NULL — does not affect existing rows)
- API: new endpoint `POST /invoices/{invoice_id}/debit-note` that creates a debit note referencing the original invoice, mirroring the existing `POST /invoices/{invoice_id}/credit-note` endpoint structure
- UBL XML: extend `ubl_generator.py` to handle `invoice_type "383"` — the structure mirrors credit notes but with `BillingReference` pointing to the original invoice
- PDF: extend PDF generator to render "DEBIT NOTE / إشعار مدين" header when `invoice_type` is "383"
- Frontend: add "Create Debit Note" button on invoice detail page alongside existing "Create Credit Note" button, only visible on ISSUED/PAID invoices

Acceptance criteria:

- Existing credit note creation is completely unaffected
 - Debit note is created with reference to original invoice
 - Debit note PDF renders correctly with correct header
 - Debit note UBL XML is valid
-

TASK 5 — Supply Date Field (P2-3)

What to build:

- DB: add `supply_date` (DATE, nullable) to invoices table. Default NULL so existing rows are unaffected.
- API: add `supply_date` as an optional field in invoice create and update request schemas. If not provided, it defaults to `issue_date`.
- PDF: render "Date of Supply" field on invoice PDF below "Invoice Date" when `supply_date` differs from `issue_date`
- UBL XML: populate `<cbc:TaxPointDate>` with `supply_date` in the UBL generator
- Frontend: add optional "Date of Supply" date picker on invoice creation form, with helper text "Leave blank if same as invoice date"

Acceptance criteria:

- Existing invoices without `supply_date` render identically to before
 - New invoices with `supply_date` show it on PDF and in XML
 - API accepts invoices without `supply_date` without error
-

TASK 6 — Arabic Invoice Fields (P2-5)

What to build:

- PDF generator: add bilingual header to all tax invoices:
 - English: "TAX INVOICE" — Arabic: "فاتورة ضريبية"
 - For simplified invoices: "SIMPLIFIED TAX INVOICE" / "فاتورة ضريبية مبسطة"
 - For credit notes: "CREDIT NOTE" / "إشعار دائن"
 - For debit notes: "DEBIT NOTE" / "إشعار مدين"
- Use a font that supports Arabic (e.g. Amiri or Noto Naskh Arabic). Add the font file to the assets folder and register it in the PDF generator.
- Arabic text must render right-to-left. Place the Arabic label to the right of the English label on the same line, or directly below it.
- Do not change any other layout, fonts, or styling of the existing invoice PDF.

- For simplified invoices, also add the statement: "VAT included / ضريبة القيمة المضافة / مشمولة"

Acceptance criteria:

- All existing invoice PDF fields render identically in position and content
- Arabic label appears on the invoice header
- Arabic text is not garbled (correct font rendering)
- Credit/debit note headers are bilingual

TASK 7 — FAF Restructure into 4 Tables + File Hash (P1-2)

What to build:

Refactor `utils/fta_audit_generator.py` to produce a FAF with the correct FTA-prescribed 4-table structure. **Keep the existing function signature so the existing API endpoint and UI continue to work without changes.**

New FAF structure (single CSV file, 4 sections separated by table headers):

Section 1 — Company Information Table

CompInfoStart

TaxablePersonNameEn, TaxablePersonNameAr, TRN, TaxAgencyName, TAN,
TaxAgentName, TAAN, PeriodStart, PeriodEnd, FAFCreationDate,
ProductVersion, FAFVersion

[one data row]

CompInfoEnd

Section 2 — Purchase Listing Table

PurcDataStart

[header row with field names]

[one row per purchase invoice line]

PurcDataEnd, PurchaseTotalAED, VATTotalAED, TransactionCountTotal

Section 3 — Supply Listing Table

SuppDataStart

[header row with field names]

[one row per sales invoice line]

SuppDataEnd, SupplyTotalAED, VATTotalAED, TransactionCountTotal

Section 4 — General Ledger Table

GLDataStart

[header row with field names]

[one row per GL entry – see Task 8 for source data]
GLDataEnd, TotalDebit, TotalCredit, TransactionCountTotal

Additional requirements:

- `ProductVersion` = "InvoLinks v{VERSION}" where VERSION is read from a config/version file
- `FAFVersion` = "FAFv1.0.0"
- `TaxAgencyName` = "InvoLinks" (the software vendor name)
- All VAT amounts must be in AED. Use `total_amount_aed` from DB where available; apply stored exchange rate for multi-currency invoices.
- Use actual `tax_code` from `invoice_line_items` table per line item — do not infer from arithmetic.
- After generating the CSV content, compute SHA-256 hash of the file content and:
 - Embed the hash as the very last line of the CSV: `FAFHash,{sha256_hex}`
 - Also generate a companion `{filename}.sha256` file containing just the hash
- Return both files as a ZIP download

Do not change:

- The existing API endpoint URL or HTTP method
- The existing UI page (FTAAuditFiles.jsx) — it should continue to trigger the same download, now receiving a ZIP instead of a raw CSV (update only the download handling if needed)

Acceptance criteria:

- Downloaded file is a ZIP containing the FAF CSV and a .sha256 file
- FAF CSV has exactly 4 sections in the correct order
- SHA-256 hash at end of CSV matches the .sha256 file
- All existing FAF data (invoices, purchases) appears in the correct sections
- Period date filter still works

TASK 8 — General Ledger / Double-Entry Accounting (P1-1)

This is the largest task. Build it carefully and incrementally.

Schema (new tables only — do not touch existing tables):

```
-- Chart of Accounts
CREATE TABLE accounts (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    company_id UUID REFERENCES companies(id),
    account_code VARCHAR(20) NOT NULL,
    account_name VARCHAR(100) NOT NULL,
    account_name_ar VARCHAR(100),
    account_type VARCHAR(20) NOT NULL, -- ASSET, LIABILITY, EQUITY, REVENUE, EXPE
    parent_account_id UUID REFERENCES accounts(id),
    is_system BOOLEAN DEFAULT FALSE, -- system accounts auto-created
    is_active BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Journal Entries (header)
CREATE TABLE journal_entries (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    company_id UUID REFERENCES companies(id),
    entry_date DATE NOT NULL,
    reference_type VARCHAR(50), -- "INVOICE", "CREDIT_NOTE", "DEBIT_NOTE", "PAYME
    reference_id UUID, -- FK to the source document
    reference_number VARCHAR(100), -- human-readable (invoice number etc.)
    description VARCHAR(500),
    is_posted BOOLEAN DEFAULT TRUE,
    created_by_user_id UUID REFERENCES users(id),
    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Journal Entry Lines (debit/credit)
CREATE TABLE journal_entry_lines (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    journal_entry_id UUID REFERENCES journal_entries(id),
    account_id UUID REFERENCES accounts(id),
    account_code VARCHAR(20), -- denormalized
    account_name VARCHAR(100), -- denormalized
    debit_amount DECIMAL(14,2) DEFAULT 0.00,
    credit_amount DECIMAL(14,2) DEFAULT 0.00,
    currency VARCHAR(3) DEFAULT 'AED',
    amount_aed DECIMAL(14,2) DEFAULT 0.00,
    description VARCHAR(500),
    line_number INTEGER
);
```

System accounts auto-created per company (on company creation or migration):

Create a function `create_default_chart_of_accounts(company_id)` that creates:

- 1100 — Accounts Receivable (ASSET)
- 2100 — Accounts Payable (LIABILITY)
- 3000 — Sales Revenue (REVENUE)
- 4000 — Purchase / Cost of Goods (EXPENSE)
- 2200 — VAT Payable (LIABILITY)
- 1200 — VAT Receivable (ASSET)
- 1000 — Cash / Bank (ASSET)

Journal entry auto-generation: When an invoice is ISSUED (status changes to ISSUED), auto-generate:

```
DR  Accounts Receivable (1100)    [invoice total incl. VAT]
    CR  Sales Revenue (3000)       [invoice subtotal excl. VAT]
    CR  VAT Payable (2200)         [VAT amount]
```

When a purchase invoice is created:

```
DR  Purchase / COGS (4000)        [purchase subtotal]
DR  VAT Receivable (1200)         [purchase VAT]
    CR  Accounts Payable (2100)    [purchase total incl. VAT]
```

When a credit note is issued:

```
DR  Sales Revenue (3000)          [credit note subtotal]
DR  VAT Payable (2200)            [credit note VAT]
    CR  Accounts Receivable (1100) [credit note total]
```

API endpoints (new, do not modify existing):

- `GET /accounts` — list chart of accounts for company
- `GET /journal-entries?from_date=&to_date=&reference_type=&page=&limit=`
- `GET /journal-entries/{id}` — entry with lines
- `GET /gl-summary?from_date=&to_date=` — account balances summary

Integration with FAF (Task 7): The GL Table in the FAF is now populated from

`journal_entry_lines` joined with `journal_entries` , mapped to the FTA FAF GL schema fields.

Migration for existing data: Write a one-time migration script

`migrations/backfill_journal_entries.py` that:

- Creates default chart of accounts for all existing companies
- Generates journal entries for all existing ISSUED/PAID invoices
- Logs progress and any skipped records

Do not change:

- Any existing invoice endpoint behaviour
- Invoice creation flow — journal entry generation is a side effect triggered after the invoice status change, wrapped in try/except so a GL failure does not break invoice issuance

Acceptance criteria:

- Issuing an invoice creates journal entry lines automatically
 - GET /journal-entries returns entries for the company
 - FAF GL Table is populated with real journal entry data
 - If journal entry creation fails, invoice issuance still succeeds (logged error only)
 - Existing companies get default chart of accounts after migration
-

TASK 9 — VAT Return File Generator (P1-3)

What to build:

New utility: `utils/vat_return_generator.py`

Data aggregation: Query invoices and expenses for the period and compute all 9 VAT return boxes:

- Box 1a–1g: Standard rated supplies per Emirate (use company emirate if not specified per invoice)
- Box 2: Supplies subject to reverse charge (`tax_code = "RC"`, output)
- Box 3: Zero-rated supplies (`tax_code = "ZR"`)

- Box 4: Supplies to GCC registered customers (tax_code = "IG")
- Box 5: Exempt supplies (tax_code = "EX")
- Box 6: Import VAT through UAE customs
- Box 7: Amendments/corrections to output
- Box 8: Standard rated expenses (input VAT recoverable)
- Box 9: Expenses subject to reverse charge (input)
- Totals: Total Output Tax, Total Input Tax, Net VAT Payable/Refundable

Excel (.xlsx) generator:

- Use openpyxl to generate a workbook matching the FTA VAT Return template layout
- Embed mandatory metadata in a hidden sheet named "Metadata":
 - Software Vendor TRN (InvoLinks TRN)
 - Software Name: "InvoLinks"
 - Software Version: read from config
 - Client TRN: the company's TRN
 - Generation timestamp
- Main sheet: "VAT Return" with all boxes filled
- Second sheet: "Transaction Detail" — one row per invoice contributing to each box, so the taxpayer can verify the totals

XML generator:

- Generate an XML file following the FTA VAT Return XML schema
- Root element: `<VATReturn>`
- Include all box values and the same metadata fields
- Sign the XML with the company's private key if available (using existing `crypto_utils.py`), otherwise leave unsigned

New API endpoint: `GET /vat-return/generate?period_start=YYYY-MM-DD&period_end=YYYY-MM-DD&format=xlsx|xml|both`

- Returns a ZIP file containing the requested format(s)

- Requires COMPANY_ADMIN or BUSINESS_ADMIN role

New UI page:

- `src/pages/VATReturn.jsx`
- Date range picker (period start / end)
- Format selector (Excel / XML / Both)
- Preview of the 9 boxes with amounts before download
- Download button
- Add link to this page in the existing sidebar navigation

Acceptance criteria:

- Excel file opens in Excel/LibreOffice without errors
 - All 9 VAT return boxes populated with correct amounts
 - Metadata sheet present with correct vendor and client details
 - XML file is well-formed
 - Existing `vat_utils.py calculate_vat_return()` function is unchanged
-

TASK 10 — Concurrent Update Prevention (P2-6)

What to build:

- Add `version` INTEGER column (default 1) to `invoices` table and `expenses` table. Existing rows get `version = 1` via migration default.
- On invoice UPDATE: require `version` field in request body. If request version $\neq$ DB version, return HTTP 409 `{"error": "Conflict: record was modified by another user. Please refresh and try again."}`. If versions match, increment version by 1 on save.
- Same pattern for expense UPDATE.
- Frontend: store the `version` field when loading an invoice/expense for edit, include it in the save request. On HTTP 409, show a user-friendly message and reload the record.

Acceptance criteria:

- Normal single-user edit flow works identically to before

- Simulated concurrent edit (two saves with same version) — second save returns 409
 - version field increments correctly on each save
-

TASK 11 — Periodic Reports (P3-1)

What to build:

New utility: `utils/report_generator.py`

Report types (all with date range filter and company_id):

1. **VAT Summary Report** — total output VAT, input VAT, net payable per period, broken down by tax code (SR/ZR/EX/RC)
2. **Sales Report** — total invoiced, total VAT collected, per customer summary
3. **Purchase Report** — total purchased, total input VAT, per supplier summary
4. **Aged Receivables Report** — outstanding invoices grouped by 0–30, 31–60, 61–90, 90+ days overdue

Export formats: PDF (using existing PDF library) and Excel (.xlsx via openpyxl)

Report periods: Accept `period_type` parameter: "weekly", "monthly", "quarterly", "annual" — auto-calculate date range from a reference date, or accept explicit `from_date/to_date`.

New API endpoints:

- GET `/reports/vat-summary?period_type=monthly&reference_date=2026-03-01&format=pdf|xlsx`
- GET `/reports/sales?period_type=quarterly&reference_date=2026-01-01&format=pdf|xlsx`
- GET `/reports/purchases?period_type=monthly&reference_date=2026-03-01&format=pdf|xlsx`
- GET `/reports/aged-receivables?as_of_date=2026-03-20&format=pdf|xlsx`

New UI page: `src/pages/Reports.jsx`

- Report type selector
- Period type selector (weekly/monthly/quarterly/annual) or custom date range
- Format selector (PDF/Excel)

- Preview summary table in the UI before download
- Download button
- **Replace the mock data in FinanceDashboard.jsx with real API calls** — this is the one existing page you should update. Do not change its layout or styling, only replace `mockData` with real fetch calls to the existing dashboard API.

Acceptance criteria:

- VAT Summary Report Excel opens without errors and shows correct totals
 - Reports match data visible in the existing invoice list
 - FinanceDashboard no longer shows hardcoded mock data
 - All existing dashboard UI elements render correctly with real data
-

TASK 12 — Data Archival & Restoration (P3-3)

What to build:

Archival:

- New API endpoint: `POST /archive/create?tax_year=2024`
 - Collects all invoices, expenses, journal entries, audit logs for the specified tax year
 - Serializes to JSON
 - Compresses to a .zip file
 - Signs the zip manifest with SHA-256
 - Stores the archive record in a new `data_archives` table:
 - `id, company_id, tax_year, file_path` (S3 key or local path), `file_size`, `sha256_hash`, `created_at`, `created_by_user_id`, `status`
 - Returns download URL for the archive ZIP
- New DB table: `data_archives` (schema above)

Restoration (read-only verification):

- New API endpoint: `POST /archive/verify` (accepts ZIP upload)
 - Recomputes SHA-256 of uploaded file

- Compares to stored hash in `data_archives`
- Returns `{"valid": true/false, "tax_year": ..., "record_count": ...}`
- Does NOT restore/overwrite live data — read-only integrity check only

Retention policy enforcement:

- New scheduled job (APScheduler or equivalent, if already used in the project):
 - Runs monthly
 - Flags any company that has data older than 5 years but no archive for that year
 - Logs a warning to the `audit_log` table with `entity_type = "RETENTION_WARNING"`

New UI section on a new `src/pages/DataArchival.jsx` page:

- List of existing archives per company (year, file size, date created, status)
- "Create Archive" button for a given tax year
- "Verify Archive" button (upload ZIP, shows verification result)
- Add link to this page in the sidebar

Acceptance criteria:

- Archive ZIP is created and downloadable
- Re-uploading the same ZIP passes verification
- A modified ZIP fails verification
- Monthly job logs retention warnings for missing archives

FINAL CHECKLIST

After all tasks are complete, verify the following still work exactly as before:

- User registration and login (including MFA)
- Invoice creation (DRAFT → ISSUED → PAID flow)
- Credit note creation referencing an invoice
- Existing FAF export download
- Existing PDF invoice generation

- UBL XML generation and digital signature
 - Role-based access control (Finance User cannot access admin endpoints)
 - Team member invitation flow
 - Subscription/plan limits enforcement
 - Existing VAT calculation in invoice line items
 - Bulk import via Excel
 - QR code on invoices
-

NOTES FOR THE AGENT

- The codebase uses FastAPI with SQLAlchemy ORM. Follow existing patterns for new endpoints (same auth decorators, same response model patterns).
- The frontend uses React with existing component library. Follow existing component patterns — do not introduce new UI frameworks.
- For Arabic text in PDFs, use the Amiri font (available via Google Fonts or PyPI `arabic-reshaper` + `python-bidi` for correct RTL rendering).
- All monetary amounts stored and exported in DECIMAL(14,2).
- All timestamps stored in UTC, displayed in the user's timezone on the frontend.
- Write a brief comment at the top of every new file explaining its purpose.
- After completing all tasks, output a summary of every file created or modified.